

Lab 02: The Interaction

Scenario 1: Association & Composition

Raw Data:

- A book titled "Java Programming" written by "John Smith". ISBN: 123-456, Price: 450 Baht. Status: Available. (3 copy)
- A book titled "OOP Concepts" written by "Will Jones". ISBN: 999-888, Price: 590 Baht. Status: Borrowed. (3 copy)
- Member: "Somsak" (Member ID: M001) wants to borrow a book.
- The Action: Somsak borrows the book "Java Programming"

Instructions:

1. **Reuse classes from Lab1 Scenario 1**
2. **Create a new Class named Member with attributes: name and memberId.**
3. **Modify the checkOut() method in the Book class (from Lab1 Scenario 1) to accept a member object as a parameter.**
 - *Example:* public void checkOut(Member borrower)
4. **Update Logic: When checkOut() is called, the book's status must change to "Borrowed" AND it should print: "Book [Title] has been borrowed by [Member Name]."**
5. **Challenge:**
 - Identification vs. Identity: If we have a new Object Book that is identical in every way (same title, same ISBN), do you think these two books are the same? And what criteria would we use to determine the 'unique identity' of an object?
 - Accountability & Responsibility: When designing the book borrowing process, should the logic to verify a member's eligibility (e.g., checking if they have reached the maximum limit of 3 books) reside within the Book class or the Member class? Please analyze which class should hold this responsibility based on OOP principles and modify the code to enforce this constraint.
6. **Submit the upgrade code and sample output**

Main class

```
package com.library.lab02;

import java.util.ArrayList;
import java.util.List;

public class LibraryManagementApp {

    public static void main(String[] args) {
        System.out.println("\n" + "=".repeat(60));
        System.out.println("  LIBRARY MANAGEMENT SYSTEM - DEMONSTRATION");
        System.out.println("=".repeat(60));

        // Instantiate two book objects based on the provided raw data
        List<Book> books = new ArrayList<>();
        for (int i = 0; i < 3; i++) {
            books.add(new Book("Java Programming", "John Smith", "123-456", 450.0,
"Available"));
            books.add(new Book("Java Programming", "John Smith", "123-456", 450.0,
"Available"));
        }

        // Instantiate member object
        Member member1 = new Member("M001", "Somsak");

        // Display initial details of both books
        System.out.println("\n--- Initial Book Information ---");
        for (Book book : books) {
            book.printSummary();
        }

        // Demonstrate checkOut() method
        System.out.println("\n--- Testing checkOut() Method ---");
        System.out.println("\nAttempting to checkout Book 1 and Book 2
(Available):");
        books.get(0).checkOut(member1);
        books.get(1).checkOut(member1);

        System.out.println("\nAttempting to checkout Book 2 again (Already
Borrowed):");
        books.get(1).checkOut(member1);
    }
}
```

```

// Demonstrate returnBook() method
System.out.println("\n--- Testing returnBook() Method ---");
System.out.println("\nReturning Book 1:");
books.get(0).returnBook();

// Challenging Scenario: Attempt to borrow more than allowed limit
System.out.println("\n--- Challenging Scenario: Borrowing Limit ---");
books.get(2).checkOut(member1);
books.get(3).checkOut(member1);
books.get(4).checkOut(member1); // This should be denied

// Display final details
System.out.println("\n--- Final Book Information ---");
for (Book book : books) {
    book.printSummary();
}
}
}

```

Example Output:

```

=====
LIBRARY MANAGEMENT SYSTEM - DEMONSTRATION
=====

--- Initial Book Information ---
Book[Title='Java Programming', Status='Available']
Book[Title='Java Programming', Status='Available']
Book[Title='Java Programming', Status='Available']
Book[Title='Java Programming', Status='Available']
Book[Title='Java Programming', Status='Available']
Book[Title='Java Programming', Status='Available']

--- Testing checkOut() Method ---

Attempting to checkout Book 1 and Book 2 (Available):
    Book 'Java Programming' has been checked out successfully.
    Book Java Programming has been borrowed by Somsak.
    Return Due Date: 2026-02-04
    Book 'Java Programming' has been checked out successfully.
    Book Java Programming has been borrowed by Somsak.
    Return Due Date: 2026-02-04

Attempting to checkout Book 2 again (Already Borrowed):
    Error: Book 'Java Programming' is already borrowed and cannot be checked out again.

```

```
--- Testing returnBook() Method ---

Returning Book 1:
  Book 'Java Programming' has been returned successfully.

--- Challenging Scenario: Borrowing Limit ---
  Book 'Java Programming' has been checked out successfully.
  Book Java Programming has been borrowed by Somsak.
  Return Due Date: 2026-02-04
  Book 'Java Programming' has been checked out successfully.
  Book Java Programming has been borrowed by Somsak.
  Return Due Date: 2026-02-04
  Member Somsak has reached the borrow limit (3).
  Borrow request denied for member Somsak.

--- Final Book Information ---
Book[Title='Java Programming', Status='Available']
Book[Title='Java Programming', Status='Borrowed']
Book[Title='Java Programming', Status='Borrowed']
Book[Title='Java Programming', Status='Borrowed']
Book[Title='Java Programming', Status='Available']
Book[Title='Java Programming', Status='Available']
```

Scenario 2: The Battle System (RPG Combat)

Concept: Understanding how objects communicate and affect each other's state.

New Requirements:

- 1. Reuse classes from Lab1 Scenario 2**
- 2. Modify the attack() Method: Instead of just printing a message, the method must now accept another Character object as a target.**
 - Method Signature: `public void attack(Character target)`
- 3. Add Damage Logic: * Add a damage attribute to the Character class or Weapon (if you have Weapon class) (e.g., Arthur: 20, Merlin: 15).**
 - When `attack(target)` is called, the Health Points (HP) of the target must decrease by the attacker's damage value.
- 4. Implement Status Checks:**
 - Add a method `isAlive()` that returns `true` if `hp > 0`.
 - Update the `displayDetails()` method to show if the character is "Active" or "Fainted".
- 5. Challenge:**
 - **Encapsulation & Data Integrity:** If you call `warrior.attack(mage)`; and the mage's HP becomes negative (e.g., -5), which method or part of the code should be responsible for preventing the HP from dropping below 0? Why is it dangerous to allow other classes to modify the `hp` attribute directly (e.g., `mage.hp = -100`)?
 - **Responsibility of the Action:** In a combat system, who should be responsible for calculating 'Defense'? Should the Attacker calculate the final damage and send it, or should the Receiver take the raw damage and subtract their own defense value? Modify your code with this reasoning using OOP principles.
- 6. Submit the upgrade code and sample output**

Main class

```
package com.rpg.lab02;

public class RPGGameApp {

    public static void main(String[] args) {
        System.out.println("\n" + "=".repeat(70));
        System.out.println("  SCENARIO 2: RPG GAME CHARACTER - DEMONSTRATION");
        System.out.println("=".repeat(70));

        // ===== CREATE WEAPONS =====
        System.out.println("\n[STEP 1] Creating Weapons...");
        Weapon swordWeapon = new Weapon("Sword", "Melee", 25, "Slash Attack");
        Weapon staffWeapon = new Weapon("Staff", "Magic", 20, "Spell Cast");
        Weapon staffHealer = new Weapon("Healing Staff", "Magic", 15, "Healing");

        // ===== CREATE CHARACTERS =====
        System.out.println("\n[STEP 2] Creating Characters...");
        Character warrior = new Character("Arthur", 10, 1500, 20, 10, swordWeapon, "Warrior");
        Character mage = new Character("Merlin", 12, 800, 15, 5, staffWeapon, "Mage");
        Healer healer = new Healer("Elara", 9, 1000, 10, 8, staffHealer, 30);

        // ===== DISPLAY INITIAL STATUS =====
        System.out.println("\n[STEP 3] Initial Character Status:");
        warrior.displayCharacterDetails();
        mage.displayCharacterDetails();
        healer.displayCharacterDetails();

        // ===== TEST ATTACK METHOD =====
        System.out.println("\n[STEP 4] Testing attack() Method:");
        System.out.println("\n--- Warrior's Turn ---");
        warrior.attack(mage);

        System.out.println("\n--- Mage's Turn ---");
        mage.attack(warrior);

        // ===== SIMULATE COMBAT =====
        System.out.println("\n[STEP 5] Status After Exchanges:");
        warrior.displayCharacterDetails();
        mage.displayCharacterDetails();
    }
}
```

```
// ===== TEST LEVEL UP =====
System.out.println("\n[STEP 6] Testing levelUp() Method:");
System.out.println("\n--- Arthur levels up! ---");
warrior.levelUp();

System.out.println("\n--- Merlin levels up! ---");
mage.levelUp();

// ===== TEST HEALER ABILITIES =====
System.out.println("\n[STEP 7] Testing Healer Special Abilities:");
System.out.println("\n--- Healer heals themselves ---");
healer.displayCharacterDetails();
healer.heal();

System.out.println("\n--- Healer heals the Warrior ---");
healer.healAlly(warrior);
}
}
```

Example Output:

```
=====
SCENARIO 2: RPG GAME CHARACTER - DEMONSTRATION
=====

[STEP 1] Creating Weapons...
[STEP 2] Creating Characters...
[STEP 3] Initial Character Status:

--- ARTHUR ---
Class: Warrior
Status: Active
Level: 10
Health Points: 1500/1500
Damage: 20
Defense: 10
Weapon: Sword (Type: Melee, Damage: 25, Ability: Slash Attack)

--- MERLIN ---
Class: Mage
Status: Active
Level: 12
Health Points: 800/800
Damage: 15
Defense: 5
Weapon: Staff (Type: Magic, Damage: 20, Ability: Spell Cast)

--- ELARA ---
Class: Healer
Status: Active
Level: 9
Health Points: 1000/1000
Damage: 10
Defense: 8
Weapon: Healing Staff (Type: Magic, Damage: 15, Ability: Healing)
Healing Power: 57

[STEP 4] Testing attack() Method:

--- Warrior's Turn ---
Arthur (Warrior) attacks Merlin with Sword!
🗡️ Raw Attack Damage: 45
🛡️ Merlin's Defense: 5 (reduces damage by 5)
💥 Actual Damage Taken: 40
❤️ Merlin's HP: 760/800

--- Mage's Turn ---
Merlin (Mage) attacks Arthur with Staff!
🪄 Raw Attack Damage: 35
🛡️ Arthur's Defense: 10 (reduces damage by 10)
💥 Actual Damage Taken: 25
❤️ Arthur's HP: 1475/1500
```

[STEP 5] Status After Exchanges:

--- ARTHUR ---

Class: Warrior
Status: Active
Level: 10
Health Points: 1475/1500
Damage: 20
Defense: 10
Weapon: Sword (Type: Melee, Damage: 25, Ability: Slash Attack)

--- MERLIN ---

Class: Mage
Status: Active
Level: 12
Health Points: 760/800
Damage: 15
Defense: 5
Weapon: Staff (Type: Magic, Damage: 20, Ability: Spell Cast)

[STEP 6] Testing levelUp() Method:

--- Arthur levels up! ---

Arthur leveled up to Level 11!
 Max Health increased to 1510 (full heal applied)

--- Merlin levels up! ---

Merlin leveled up to Level 13!
 Max Health increased to 810 (full heal applied)

[STEP 7] Testing Healer Special Abilities:

--- Healer heals themselves ---

--- ELARA ---

Class: Healer
Status: Active
Level: 9
Health Points: 1000/1000
Damage: 10
Defense: 8
Weapon: Healing Staff (Type: Magic, Damage: 15, Ability: Healing)
Healing Power: 57
Elara (Healer) casts a healing spell!
✦ Healing Amount: 57 (Base: 30 + Level Bonus: 27)
♥ Restored 57 HP! Current HP: 1000/1080

--- Healer heals the Warrior ---

Elara (Healer) casts healing spell on Arthur!
✦ Healing Amount: 57
♥ Restored 57 HP to Arthur! Their HP: 1510